

Práctica 2 – Algoritmo RSA

Ulises Zarate Concha

Seguridad Informática · UACAM · Facultad de Ingeniería

1. Datos de entrada

$p = 67$

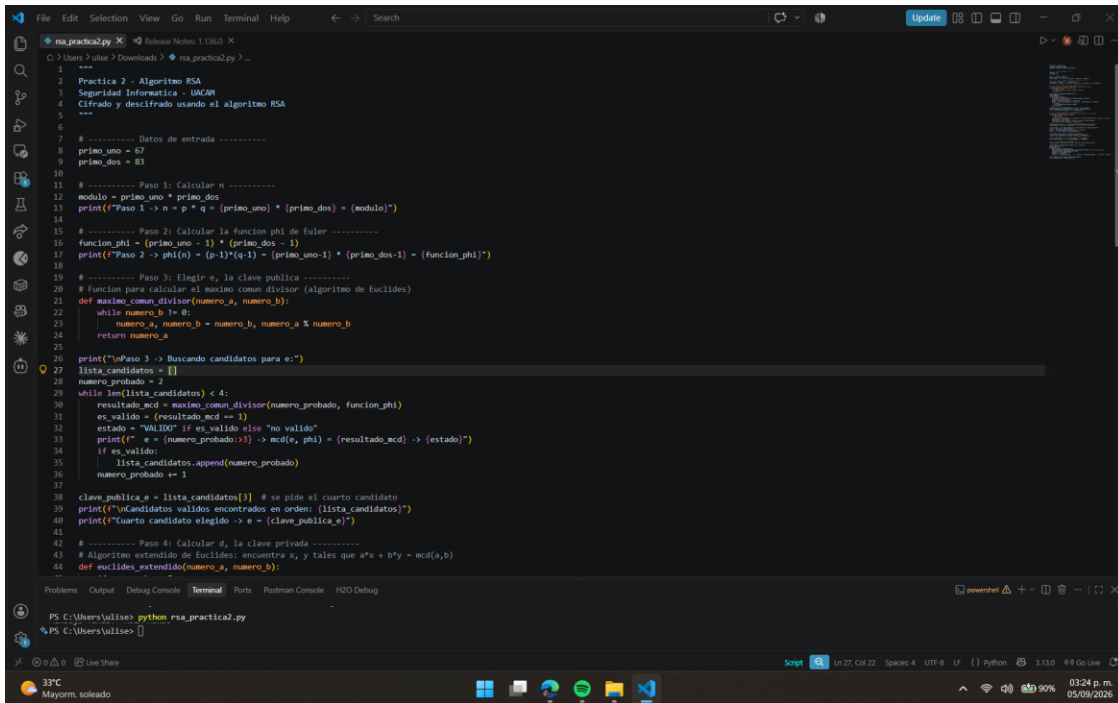
$q = 83$

2. Resumen de resultados

p	67
q	83
$n = p \times q$	5561
$\phi(n) = (p-1)(q-1)$	5412
Candidatos válidos de e (en orden)	5, 7, 13, 17
e elegido (4to candidato)	17
d (clave privada)	4457
Comprobación ($e \times d \bmod \phi(n)$)	1
Clave pública	($e = 17, n = 5561$)
Clave privada	($d = 4457, n = 5561$)
Mensaje descifrado	"HOLA MUNDO"

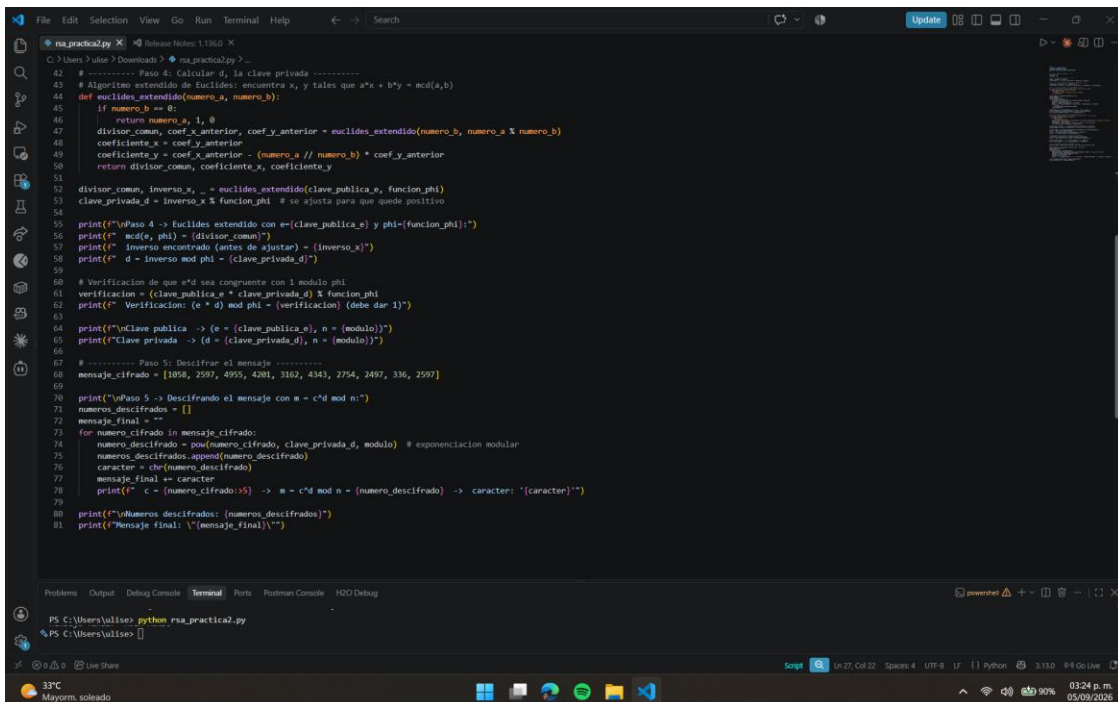
3. Evidencias — Código fuente

Script en Python (rsa_practica2_es.py) que implementa el algoritmo de Euclides, el algoritmo extendido de Euclides y el descifrado RSA.



```
1 #
2 Practica 2 - Algoritmo RSA
3 Seguridad Informatica - UACM
4 Cifrado y descifrado usando el algoritmo RSA
5 #
6
7 # ----- Datos de entrada -----
8 primo_uno = 67
9 primo_dos = 83
10
11 # ----- Paso 1: Calcular n -----
12 modulo = primo_uno * primo_dos
13 print(f"Paso 1 -> n = p * q = {primo_uno} * {primo_dos} = {modulo}")
14
15 # ----- Paso 2: Calcular la funcion phi de Euler -----
16 funcion_phi = (primo_uno - 1) * (primo_dos - 1)
17 print(f"Paso 2 -> phi(n) = (p-1)*(q-1) = {primo_uno-1} * {primo_dos-1} = {funcion_phi}")
18
19 # ----- Paso 3: Elegir e, la clave publica -----
20 # Funcion para calcular el maximo comun divisor (algoritmo de Euclides)
21 def maximo_comun_divisor(numero_a, numero_b):
22     while numero_b != 0:
23         numero_a, numero_b = numero_b, numero_a % numero_b
24     return numero_a
25
26 print(f"Paso 3 -> Buscando candidatos para e:")
27 lista_candidatos = []
28 numero_probado = 2
29 while len(lista_candidatos) < 4:
30     resultado_mcd = maximo_comun_divisor(numero_probado, funcion_phi)
31     es_valido = (resultado_mcd == 1)
32     estado = "VALIDO" if es_valido else "No valido"
33     print(f"e = {numero_probado:3} -> mcd(e, phi) = {resultado_mcd} -> {estado}")
34     if es_valido:
35         lista_candidatos.append(numero_probado)
36     numero_probado += 1
37
38 clave_publica_e = lista_candidatos[3] # se pide el cuarto candidato
39 print(f"Candidatos validos encontrados en orden: {lista_candidatos}")
40 print(f"Cuarto candidato elegido -> e = {clave_publica_e}")
41
42 # ----- Paso 4: Calcular d, la clave privada -----
43 # Algoritmo extendido de Euclides: encuentra x, y tales que a*x + b*y = mcd(a,b)
44 def euclides_extendido(numero_a, numero_b):
45     # ... (código omitido) ...
46
47 divisor_comun, inverso_x, _ = euclides_extendido(clave_publica_e, funcion_phi)
48 clave_privada_d = inverso_x % funcion_phi # se ajusta para que quede positivo
49
50 print(f"Paso 4 -> Euclides extendido con e={clave_publica_e} y phi={funcion_phi}")
51 print(f"mcd(e, phi) = {divisor_comun}")
52 print(f"inverso encontrado (antes de ajustar) = {inverso_x}")
53 print(f"d = inverso mod phi = {clave_privada_d}")
54
55 # Verificación de que e*d sea congruente con 1 modulo phi
56 verificacion = (clave_publica_e * clave_privada_d) % funcion_phi
57 print(f"Verificación: (e * d) mod phi = {verificacion} (debe dar 1)")
58
59 print(f"Clave publica -> e = {clave_publica_e}, n = {modulo}")
60 print(f"Clave privada -> d = {clave_privada_d}, n = {modulo}")
61
62 # ----- Paso 5: Descifrar el mensaje -----
63 mensaje_cifrado = [1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597]
64
65 print(f"Paso 5 -> Descifrando el mensaje con m = c^d mod n")
66 numeros_descifrados = []
67 mensaje_final = ""
68 for numero_cifrado in mensaje_cifrado:
69     numero_descifrado = pow(numero_cifrado, clave_privada_d, modulo) # exponenciacion modular
70     numeros_descifrados.append(numero_descifrado)
71     caracter = chr(numero_descifrado)
72     mensaje_final += caracter
73     print(f" c = {numero_cifrado:5} -> m = c^d mod n = {numero_descifrado} -> caracter: '{caracter}'")
74
75 print(f"Numeros descifrados: {numeros_descifrados}")
76 print(f"Mensaje final: '{mensaje_final}'")
```

Figura 1. Código fuente del script en Python (parte 1) — cálculo de n , $\phi(n)$ y búsqueda de candidatos para e .



```
42 # ----- Paso 4: Calcular d, la clave privada -----
43 # Algoritmo extendido de Euclides: encuentra x, y tales que a*x + b*y = mcd(a,b)
44 def euclides_extendido(numero_a, numero_b):
45     if numero_b == 0:
46         return numero_a, 1, 0
47     divisor_comun, coef_x_anterior, coef_y_anterior = euclides_extendido(numero_b, numero_a % numero_b)
48     coeficiente_x = coef_x_anterior
49     coeficiente_y = coef_x_anterior - (numero_a // numero_b) * coef_y_anterior
50     return divisor_comun, coeficiente_x, coeficiente_y
51
52 divisor_comun, inverso_x, _ = euclides_extendido(clave_publica_e, funcion_phi)
53 clave_privada_d = inverso_x % funcion_phi # se ajusta para que quede positivo
54
55 print(f"Paso 4 -> Euclides extendido con e={clave_publica_e} y phi={funcion_phi}")
56 print(f"mcd(e, phi) = {divisor_comun}")
57 print(f"inverso encontrado (antes de ajustar) = {inverso_x}")
58 print(f"d = inverso mod phi = {clave_privada_d}")
59
60 # Verificación de que e*d sea congruente con 1 modulo phi
61 verificacion = (clave_publica_e * clave_privada_d) % funcion_phi
62 print(f"Verificación: (e * d) mod phi = {verificacion} (debe dar 1)")
63
64 print(f"Clave publica -> e = {clave_publica_e}, n = {modulo}")
65 print(f"Clave privada -> d = {clave_privada_d}, n = {modulo}")
66
67 # ----- Paso 5: Descifrar el mensaje -----
68 mensaje_cifrado = [1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597]
69
70 print(f"Paso 5 -> Descifrando el mensaje con m = c^d mod n")
71 numeros_descifrados = []
72 mensaje_final = ""
73 for numero_cifrado in mensaje_cifrado:
74     numero_descifrado = pow(numero_cifrado, clave_privada_d, modulo) # exponenciacion modular
75     numeros_descifrados.append(numero_descifrado)
76     caracter = chr(numero_descifrado)
77     mensaje_final += caracter
78     print(f" c = {numero_cifrado:5} -> m = c^d mod n = {numero_descifrado} -> caracter: '{caracter}'")
79
80 print(f"Numeros descifrados: {numeros_descifrados}")
81 print(f"Mensaje final: '{mensaje_final}'")
```

Figura 2. Código fuente del script en Python (parte 2) — cálculo de d con el algoritmo extendido de Euclides y descifrado del mensaje.

4. Evidencias — Ventana de resultados

Salida obtenida al ejecutar el script con el comando: `python rsa_practica2_es.py`

```

PS C:\Users\ulise> python rsa_practica2.py
Paso 1 -> n = p * q = 67 * 83 = 5561
Paso 2 -> phi(n) = (p-1)*(q-1) = 66 * 82 = 5412

Paso 3 -> Buscando candidatos para e:
e = 2 -> mcd(e, phi) = 2 -> no valido
e = 3 -> mcd(e, phi) = 3 -> no valido
e = 4 -> mcd(e, phi) = 4 -> no valido
e = 5 -> mcd(e, phi) = 1 -> VALIDO
e = 6 -> mcd(e, phi) = 6 -> no valido
e = 7 -> mcd(e, phi) = 1 -> VALIDO
e = 8 -> mcd(e, phi) = 4 -> no valido
e = 9 -> mcd(e, phi) = 3 -> no valido
e = 10 -> mcd(e, phi) = 2 -> no valido
e = 11 -> mcd(e, phi) = 11 -> no valido
e = 12 -> mcd(e, phi) = 12 -> no valido
e = 13 -> mcd(e, phi) = 1 -> VALIDO
e = 14 -> mcd(e, phi) = 2 -> no valido
e = 15 -> mcd(e, phi) = 3 -> no valido
e = 16 -> mcd(e, phi) = 4 -> no valido
e = 17 -> mcd(e, phi) = 1 -> VALIDO

Candidatos validos encontrados (en orden): [5, 7, 13, 17]
Cuarto candidato elegido -> e = 17

Paso 4 -> Euclides extendido de e=17 y phi=5412:
mcd(e, phi) = 1
x (inverso antes de ajustar) = -955
d = x mod phi = 4457
Comprobacion: (e * d) mod phi = 1 (debe ser 1)

Clave publica -> (e = 17, n = 5561)
Clave privada -> (d = 4457, n = 5561)

Paso 5 -> Descifrando mensaje con m = c^d mod n:

```

Figura 3. Salida de la terminal (parte 1) — cálculo de n , $\phi(n)$, búsqueda de los 4 candidatos de e y cálculo de d mediante Euclides extendido.

```

Paso 5 -> Descifrando mensaje con m = c^d mod n:
c = 1058 -> m = c^d mod n = 72 -> caracter: 'H'
c = 2597 -> m = c^d mod n = 79 -> caracter: 'O'
c = 4955 -> m = c^d mod n = 76 -> caracter: 'L'
c = 4201 -> m = c^d mod n = 65 -> caracter: 'A'
c = 3162 -> m = c^d mod n = 32 -> caracter: ' '
c = 4343 -> m = c^d mod n = 77 -> caracter: 'M'
c = 2754 -> m = c^d mod n = 85 -> caracter: 'U'
c = 2497 -> m = c^d mod n = 78 -> caracter: 'N'
c = 336 -> m = c^d mod n = 68 -> caracter: 'D'
c = 2597 -> m = c^d mod n = 79 -> caracter: 'O'

Numeros descifrados: [72, 79, 76, 65, 32, 77, 85, 78, 68, 79]
Mensaje final: "HOLA MUNDO"
PS C:\Users\ulise>

```

Figura 4. Salida de la terminal (parte 2) — descifrado carácter por carácter del mensaje, resultando en "HOLA MUNDO".